

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Universitatea Politehnică din București

Arhitectură neuronală analogică în inversie slabă pentru convoluție directă în aplicații CNN

student: Ghinea Valentin-Andrei, 434E

profesor coordonator: Marius Enăchescu

Sesiunea de comunicări științifice 2025-2026

Rețele neuronale convoluționale (CNN)

Rețelele neuronale convoluționale sunt un caz particular de rețele neuronale care folosesc produsul de convoluție drept operație ce stă la baza calcului activărilor în cel puțin un strat intern. Acestea sunt utilizate pe scară largă în aplicații de viziune computerizată. De obicei, acestea sunt folosite atunci când setul de date este de tip tablou sau matrice. Procesarea imaginilor, detecția de margini și trasaturi semnificative (obiecte) sunt printre cele mai comune utilizări ale rețelelor convoluționale.[1]

Un CNN constă în mai multe straturi convoluționale care reduc dimensionalitatea setului de date extragând trasaturile țintă printr-o funcție specifică. Întreaga arhitectură presupune mai multe straturi și operații: stratul de intrare, straturile de **convoluție** pentru calculul activărilor și funcția de activare neliniară, stratul de netezire a datelor, rețea FCNN etc. [3] [4]

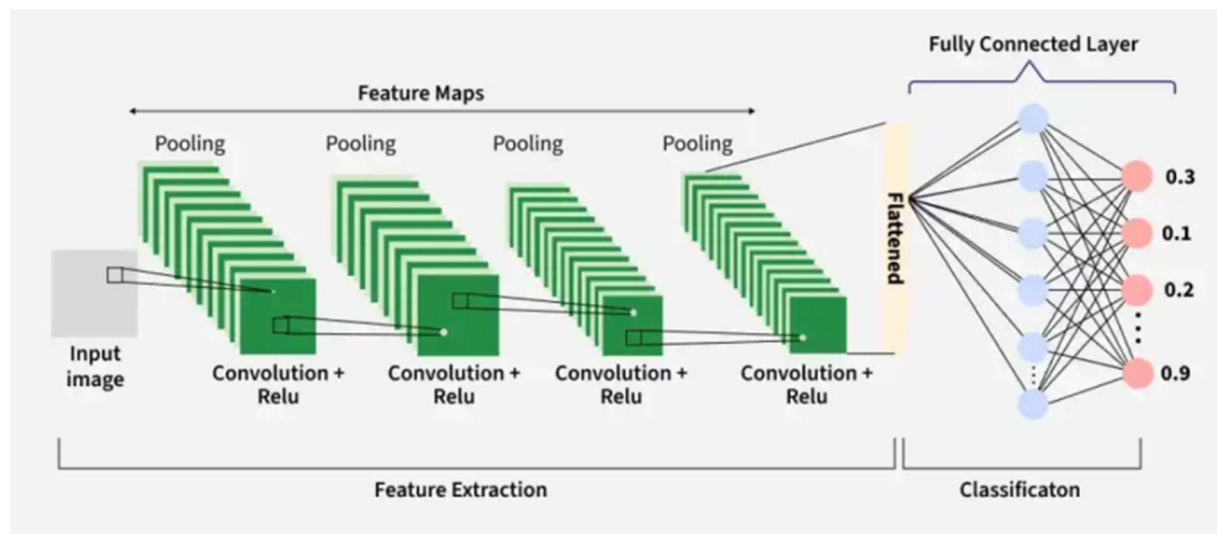


Fig.1

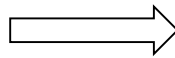
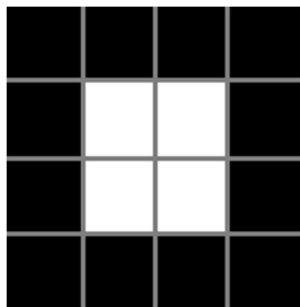
Convolutional Neural Networks

(sursa: www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/)

În această lucrare este prezentată o implementare analogică pentru calculul convoluțional implicat în arhitectura CNN și are ca scop extracția trăsăturilor țintă în mod direct prin **interfațare analogică**.

Operația de convoluție asupra unei imagini

Pentru a putea realiza produsul de convoluție asupra unui segment de imagine, acestea trebuie reprezentate matricial.



$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Fig. 2

În această lucrare, imaginile vor fi reprezentate în tonuri gri. Valoarea fiecărui pixel fiind $x_{i,j} \in [0,1]$; unde “0” reprezintă **negru** absolut, iar “1” reprezintă **alb** absolut. Cromatica nu este foarte relevantă pentru aplicații de edge detection și detecție de obiecte simple.

Alegerea nu este întâmplătoare, deoarece fiecare pixel poate fi reprezentat în acest fel printr-o valoare **analogică** (în acest caz: o tensiune). Practic, nu este nevoie de **cuantizarea** mărimii de intrare care poate fi un semnal analogic recepționat în mod direct fără conversie **analog-digital** explicită. Fiecare pixel va reprezenta de exemplu intensitatea luminoasă în mod direct (normalizat).

Considerăm imaginea de intrare ca fiind matricea $I_{N \times M}$ operatorul kernel specific K_k , matrice patratică de cele mai multe ori.

În funcție de aplicații, kernelul K poate lua diferite valori. Sunt cunoscute Sobel kernel, Prewitt kernel și filtrul Laplacian în aplicațiile de detecție de margini.

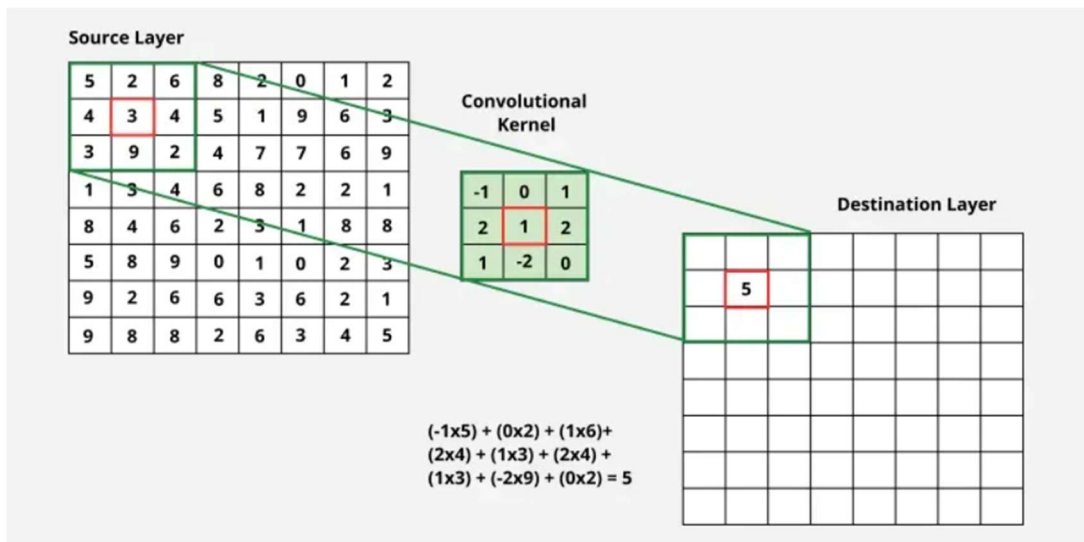


Fig. 3

operația de convoluție

(sursa: <https://www.geeksforgeeks.org/deep-learning/kernels-filters-in-convolutional-neural-network/>)

$$Y_{(i,j)} = (K * X)_{(i,j)} \sum_m \sum_n X(i + m, j + n) K(m, n) \quad (1)$$

$$m, n \in [0, 2] ; i \in [0, N-2], j \in [0, M-2].$$

De fapt, kernelul K parcurge matricea imaginii efectuându-se câte o convoluție pentru fiecare pixel central. In acest mod, in funcție de tipul kernelului pot fi extrase diferite trăsături specifice. (imagini de la: [Introduction to Convolution Neural Network - GeeksforGeeks](#))

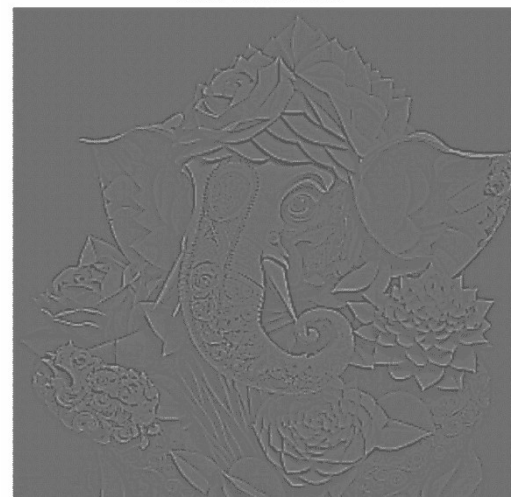
Original Image Shape: (300, 300, 1)

Original Image



After Convolution Shape: (1, 300, 300, 1)

After Convolution



$$K = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

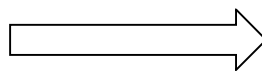


Fig. 4

Arhitectura propusa pentru calculul produsului de convoluție

Pentru a realiza operația de multiplicare-acumulare (convoluție) se propune o arhitectura analogică alcătuită din celule de calcul care efectuează **produsul** și rețea de însumare pentru **acumulare**.

Celula analogica de multiplicare:

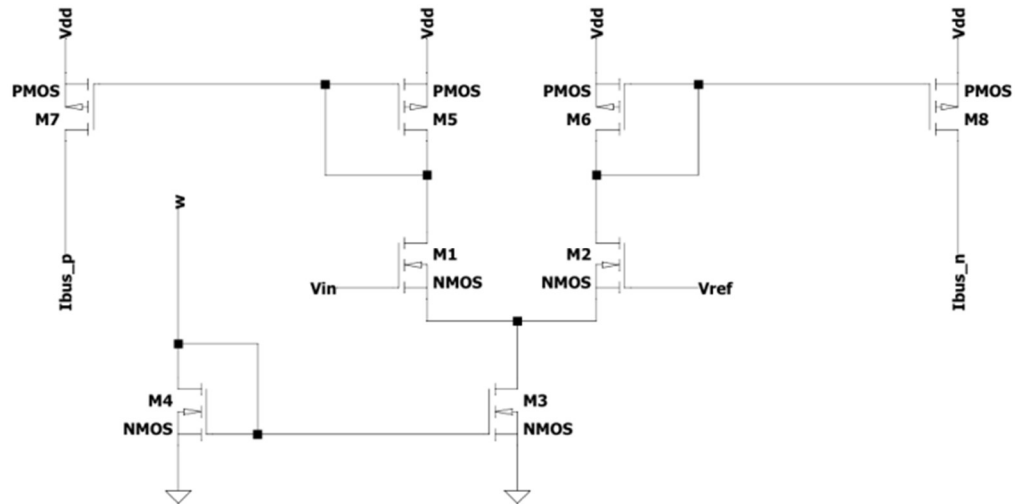


Fig. 5

Etajul diferențial va opera în regim de inversie slabă. Avantajele unei astfel de abordări sunt: consumul redus - zeci de nA și capacitatea de a realiza funcția $f(x) = wx$, necesară în calculul convoluției (w și k reprezintă același lucru).

Pentru ca tranzistorii M1, M2 să opereze în regim de inversie slabă aceștia trebuie să respecte condiția aproximativă pentru panta $\frac{gm}{Id} \approx [20,30]V^{-1}$. O altă condiționare este curentul de bias pentru sursa perechii diferențiale: zeci de nA.

Funcționarea corectă presupune ca ieșirea notată cu y să fie liniar dependentă de x , adică de V_{in} .

Conform ecuațiilor pentru modelul tranzistorului în inversie slabă (în saturație):

$$I_d = I_s \exp\left(\frac{V_{gs} - V_{th}}{nV_t}\right) \quad [2] \quad (2)$$

$$I_{d1} + I_{d2} = I_{tail}$$

Conform ecuației (2), curenții de drenă ai tranzistorilor de intrare M1 și M2 sunt:

$$I_{d1} = I_s \exp\left(\frac{V_{in} - V_{th}}{nV_t}\right)$$

$$I_{d2} = I_s \exp\left(\frac{V_{ref} - V_{th}}{nV_t}\right)$$

Not. $V_{in} - V_{ref} = x$ și $nV_t = \gamma$. Prin urmare:

$$I_{d1} = I_{d2} \exp\left(\frac{x}{\gamma}\right)$$

La semnal mic, $I_{d1} - I_{d2} = \Delta I$. Prin urmare:

$$\Delta I = I_{tail} \left(\frac{\exp\left(\frac{x}{\gamma}\right) - 1}{1 + \exp\left(\frac{x}{\gamma}\right)} \right)$$

$$\Delta I = I_{tail} \tanh\left(\frac{x}{2\gamma}\right)$$

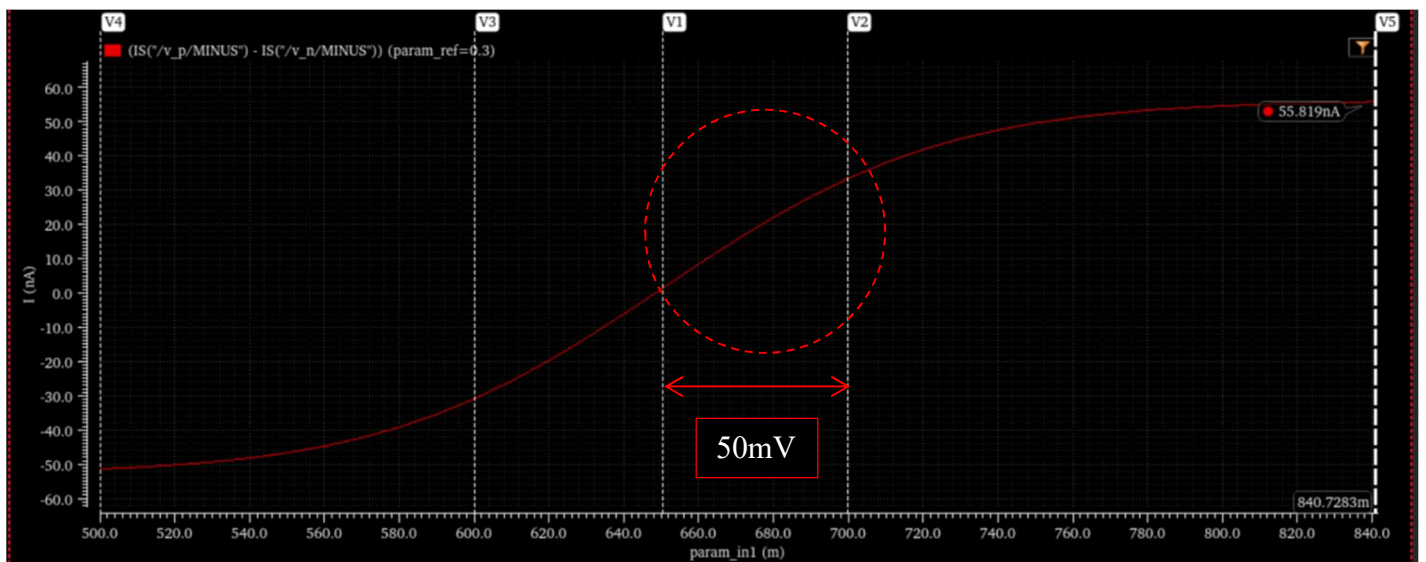
Pentru $x \ll 2\gamma$, $\Delta I = \frac{I_{tail}}{2\gamma} x$. Reprezintă de fapt expresia: $I_{out} = g_{m,diff} V_{diff}$

Un dezavantaj important este că g_m este dependent direct de temperatura prin tensiunea termică. Acest fapt este totuși neglijat pentru moment deoarece întreaga rețea este afectată într-o primă aproximare de variații de temperatură uniforme rezultând în apariția unui bias al valorilor de ieșire.

Expresia de mai sus se traduce tocmai în $y = w x$ unde x este valoarea curentă a unui pixel iar w este ponderea asociată. Fiecare pondere reprezintă de fapt elementele kernelului K .

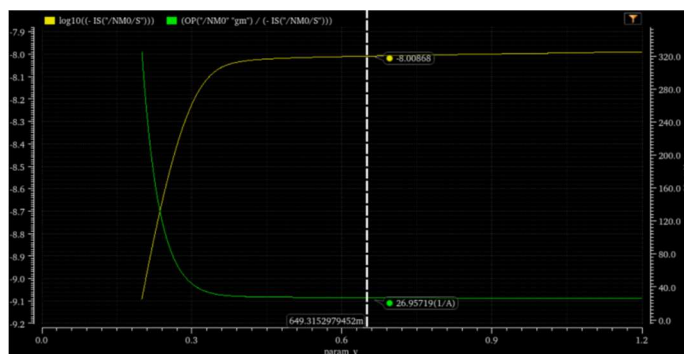
Pentru a păstra proprietatea de liniaritate este necesar ca $x \ll 2 \gamma$. $\gamma = nVt$. Prin urmare variațiile tensiunii de intrare $V_{in} \in [0, 50]$ mV cu aproximație. Valorile pixelilor de intrare aflate între 0 și 1 vor fi scalate în intervalul 0:50 mV. Unui pixel pur alb îi va corespunde **tensiunea de intrare diferențială** de 50mV, celui pur negru, tensiune diferențială nulă și astfel, toate valorile continue în intervalul 0:50 mV.

O posibilă tratare diferită a funcției caracteristice a etajului **tanh** (tangenta hiperbolică) este utilizarea valorilor de intrare în afara intervalului de **liniaritate** și **exploatarea neliniarității** în calcularea funcțiilor de activare din rețeaua neuronală.



$\Delta I = f(V_{in})$ Fig. 6

Se va folosi doar intervalul pozitiv de liniaritate pentru ca valorile de intrare trebuie să fie strict pozitive față de tensiunea de mod comun $V_{CM} = 650$ mV. Tranzistorii celulei funcționează în regim de inversie slabă, iar g_m (ponderea w) este cea dorită: $g_m \approx 25 V^{-1}$.



Dimensionarea tranzistorilor presupune respectarea tuturor condițiilor menționate mai sus.

Fig. 7

Liniaritatea celulei se verifica atât ca funcție de pondere, $y = f(w)$ cât și ca funcție de x , $y = g(x)$. Cu alte cuvinte, variația **Itail** modifică liniar iesirea (respecta **proprietatea de pondere**), la fel și variația tensiunii diferențiale aplicate asupra celulei.

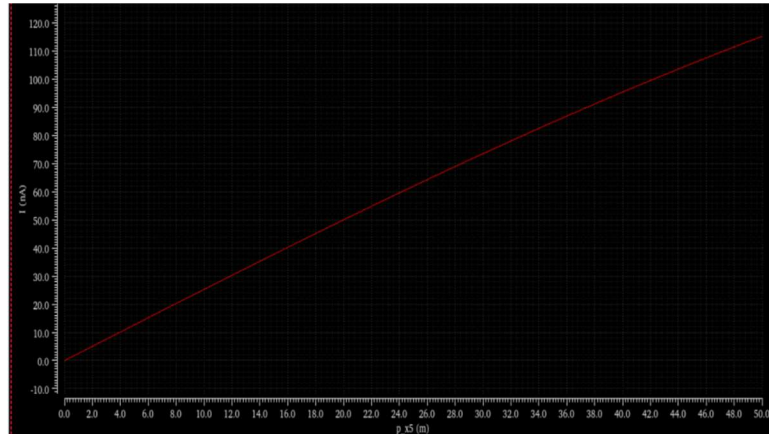


Fig. 8

$$\Delta I = f(I_{tail})$$

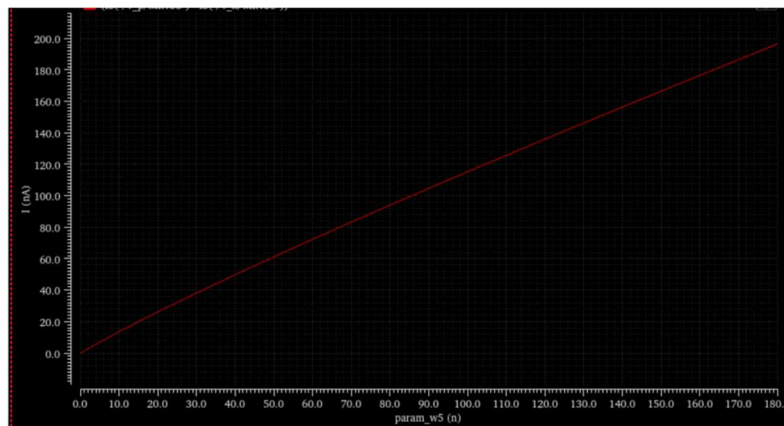


Fig. 9

$$\Delta I = g(V_{in})$$

Exemplificare: Un astfel de etaj diferențial reprezintă celula de multiplicare. Pentru ușurință în calcule, kernelul folosit va fi de dimensiune 3x3, de tip Laplacian.

$$K = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

Fie $X_{5 \times 5}$ matricea imaginii de intrare:

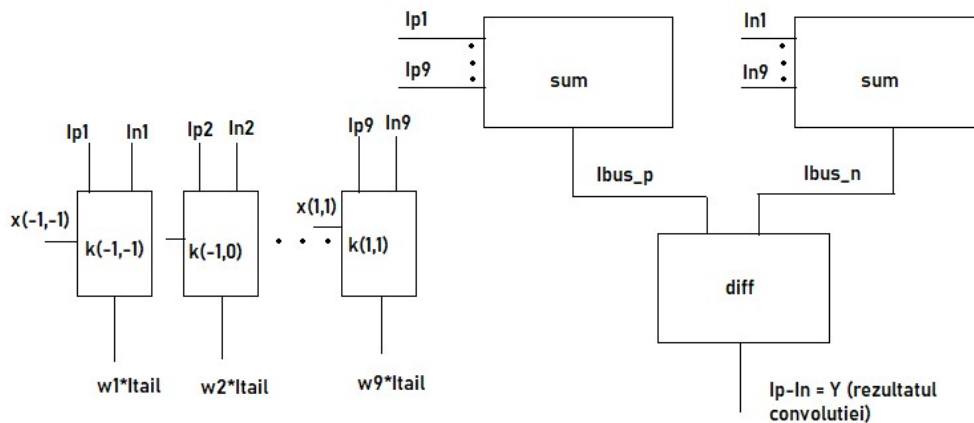
$$X = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

$X_{i,j}$, $i = \{0,4\}$ si $j = \{0,4\}$ sunt pixelii de **padding** care asigură suprapunerea kernelului în produsul de convoluție asupra pixelilor de interes.

$$X = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad \text{fig.10}$$

$$X1 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} x(-1,-1) & x(-1,0) & x(-1,1) \\ x(0,-1) & x(0,0) & x(0,1) \\ x(1,-1) & x(1,0) & x(1,1) \end{pmatrix}$$

$$Y1 = \sum_{m=-1}^1 \sum_{n=-1}^1 X(m,n)K(m,n) \quad (3)$$



Implementarea blocurilor funcționale se face astfel:

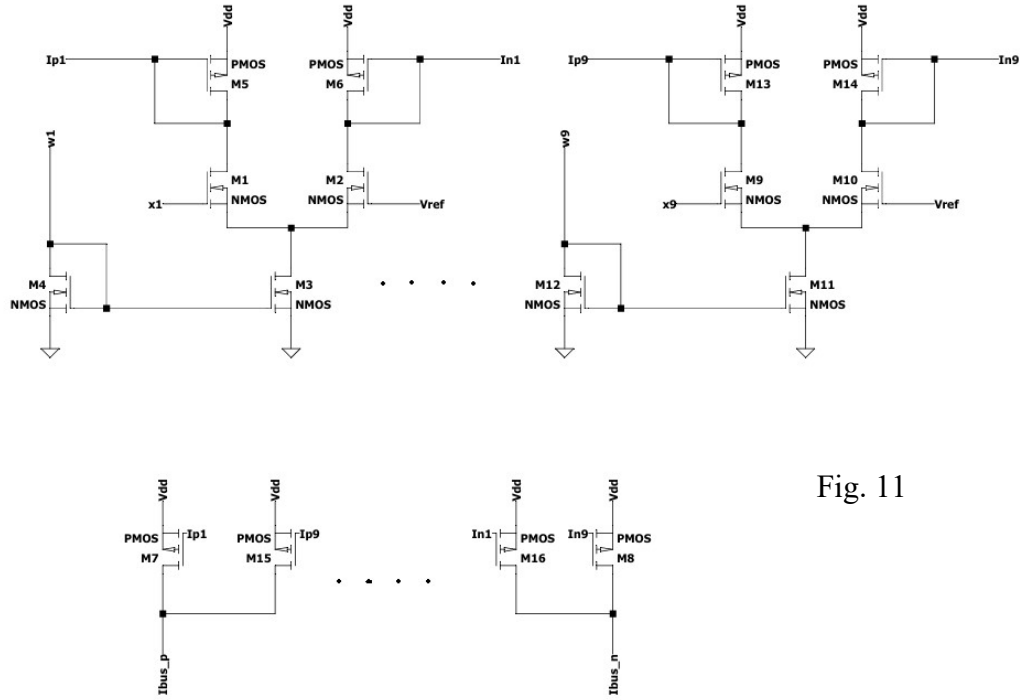


Fig. 11

$$I_{bus_p} = I_{p1} + I_{p2} + \dots + I_{p9}, \quad I_{bus_n} = I_{n1} + I_{n2} + \dots + I_{n9}.$$

Conform ecuației $\Delta I = \frac{I_{tail}}{2\gamma} x$, unde ΔI reprezintă $I_p - I_n$ și x reprezintă V_{in} :

$I_{tail} = I_0 w$, unde I_0 reprezintă curentul de coadă corespunzător ponderii de baza $w = 1$ (în această arhitectură este ales $I_0 = 20nA$ pentru a respecta condițiile de polarizare menționate mai sus). Notăm $I_0/(2\gamma) = g_{m,0} = \beta$.

$\Delta I = \frac{I_{tail}}{2\gamma} x = w \beta x$. Cu toate acestea, β este un factor neglijabil putând fi considerat un factor de câștig uniform în toate operațiile efectuate în calculul produsului de convoluție.

Prin urmare, $Y = I_{bus_p} - I_{bus_n} = \beta \sum_{i=1}^9 w(i)x(i)$. Aceasta formă este perfect echivalentă cu ecuația (3).

$$\beta \sum_{i=1}^9 w(i)x(i) = \beta Y(j), \text{ cu } j \in [1,9] \quad (4)$$

Valoarea Y obtinuta anterior este rezultatul convoluției între X_1 și K . Pentru a aplica kernelul asupra întregii imagini este nevoie de $(N - 2)^2$ operații în cazul matricii pătratică de intrare considerate anterior. Astfel, este nevoie de 9 astfel de blocuri de multiplicare-acumulare. Sunt două implementări posibile, fiecare optimizând un aspect al arhitecturii. Dacă se dorește paralelism perfect, va fi nevoie de $(N - 2)^2$ blocuri, fiecare conținând K^2 celule fundamentale pentru un kernel de tip pătratic și de dimensiune K . Fiecare celulă conține cel puțin 5 tranzistori.

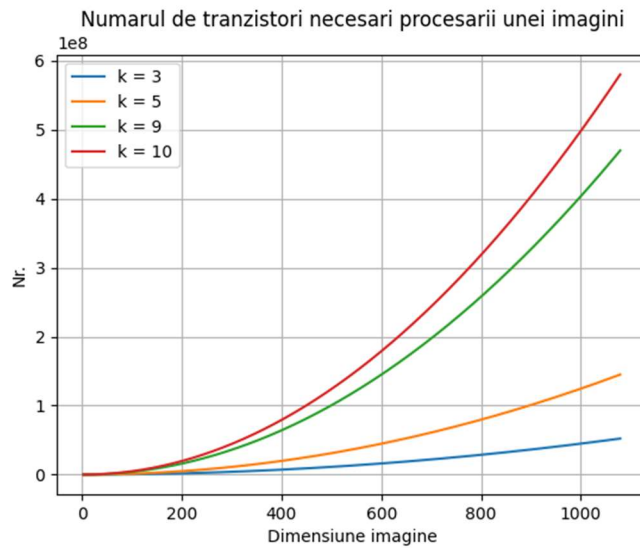


Fig. 12

Este evident faptul că această implementare se pretează la kerneluri de dimensiuni mici, $K = \{3, 5\}$. Pentru kerneluri mari, numărul tranzistorilor necesari este foarte mare, iar robustetea arhitecturii are de suferit semnificativ.

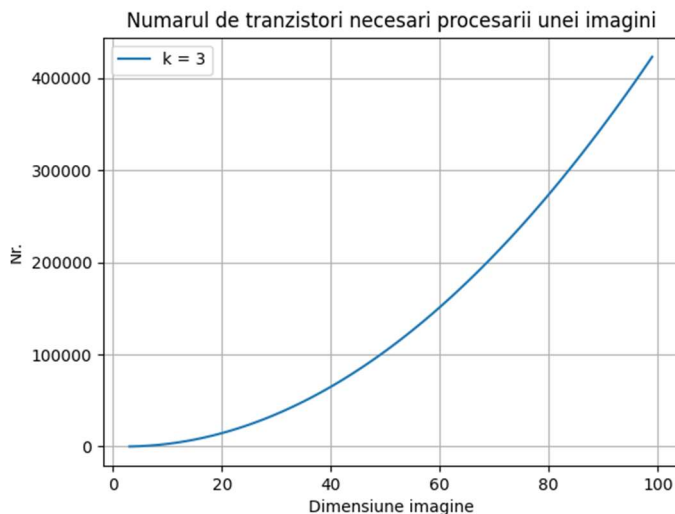


Fig. 13

Pentru a reduce numărul de tranzistori necesari și pentru a permite prelucrarea unor imagini de dimensiuni mai mari se propune o alternativă la implementarea pur paralelă. Se va folosi un sir de P blocuri de convolutie, unde $P = N$, numărul de linii din matricea de intrare sau $P = M$, numărul de coloane din matricea de intrare. Aceasta implementare necesita un sistem mai complex de rerutare a intrarilor, devenind astfel un sistem hibrid (digital-analog). Avantajul este ca reduce semnificativ numărul de tranzistori necesari crescand dimensionalitatea acceptată a matricii de intrare, dar și a timpului de procesare.

Astfel, numărul de tranzistori necesari (referitor exclusiv la blocurile de procesare) devine: $Nr = N K^2$.

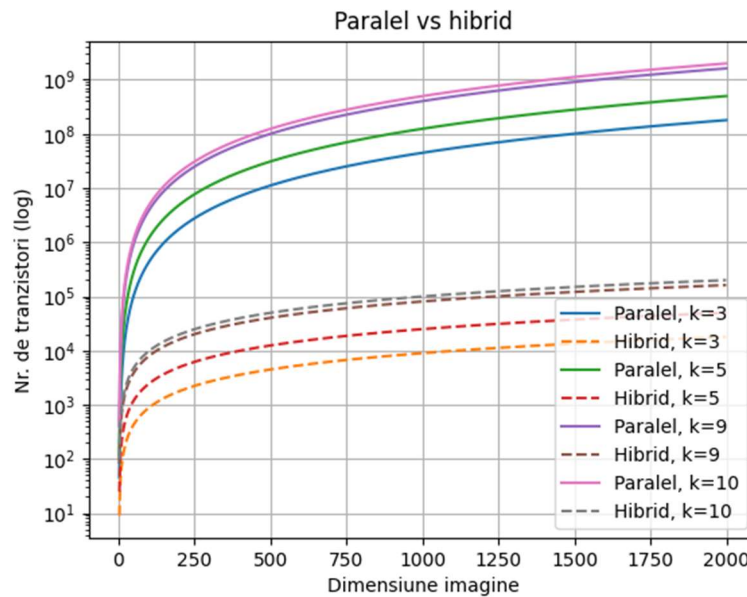


Fig. 14

Verificarea rezultatelor arhitecturii s-a realizat in mediul de simulare Cadence Virtuoso – maestro (detalii despre implementare – **Anexa**).

Revenind asupra exemplificarii anterioare:

Fie:

$$K = \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$$

$$X = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Rezultatul matematic al produsului de convolutie (in sensul descris in ecuatia (1)) $\mathbf{X} * \mathbf{K} = \mathbf{Y}$ este:

$$Y = \begin{pmatrix} -2 & 7 & -2 \\ -3 & 6 & -3 \\ -2 & 7 & -2 \end{pmatrix}$$

Semnul minus al răspunsului reprezinta tranzitia catre zone mai intunecate. In aplicatiile de detectie de margini nu este imperativ necesar ca outputul să prezinte semn. Prin urmare, in această propunere de arhitectura răspunsul va fi mereu $|Y|$.

In ceea ce priveste semnul minus din interiorul kernelului acesta va fi implementat prin inversarea polaritații celulei corespunzătoare elementului negativ. Pentru $k(-1,-1)$: I_n se va insuma in I_{bus_p} si I_p se va insuma in I_{bus_n} . În acest fel, $\Delta I(-1,-1) = -w_{-1,-1} x_{-1,-1}$.

Rezultatul obținut în simulare este:

$$\Delta I = \begin{pmatrix} 52 & 150 & 52 \\ 78 & 124 & 78 \\ 52 & 150 & 52 \end{pmatrix} nA$$

Dar,

$$\Delta I = \begin{pmatrix} \beta Y(1) & \beta Y(2) & \beta Y(3) \\ \beta Y(4) & \beta Y(5) & \beta Y(6) \\ \beta Y(7) & \beta Y(8) & \beta Y(9) \end{pmatrix} nA = \beta \begin{pmatrix} Y(1) & Y(2) & Y(3) \\ Y(4) & Y(5) & Y(6) \\ Y(7) & Y(8) & Y(9) \end{pmatrix} nA$$

$$Y = \frac{1}{\beta} \Delta I$$

$$Y = \begin{pmatrix} 2.6 & 7.5 & 2.6 \\ 3.9 & 6.2 & 3.9 \\ 2.6 & 7.5 & 2.6 \end{pmatrix}$$

Pana acum, am verificat liniaritatea celulei diferentiale de calcul in regim de subconductie. Trebuie verificata liniaritatea si proprietatea de multiplicare-acumulare a sistemului intreg. Pentru aceasta s-au izolat 3 celule care vor trebui sa indeplineasca functia de suma de produse asupra unui sir complet de coduri de intrare.

Fie X sirul de intrare si Y outputul celulelor:

$$X = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix} \text{ si } K = \begin{pmatrix} 4 \\ 2 \\ 1 \end{pmatrix} \rightarrow Y = \begin{pmatrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \end{pmatrix}$$

$$Y = X K, \text{ produs scalar}$$

Aplicat asupra sistemului analogic:

$$V_{in} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 50 \\ 0 & 50 & 0 \\ 0 & 50 & 50 \\ 50 & 0 & 0 \\ 50 & 0 & 50 \\ 50 & 50 & 0 \\ 50 & 50 & 50 \end{pmatrix} mV, K = \begin{pmatrix} 80 \\ 40 \\ 20 \end{pmatrix} nA \rightarrow \Delta I = \begin{pmatrix} 0 \\ 26.22 \\ 49.91 \\ 76.14 \\ 94.16 \\ 120.38 \\ 144.08 \\ 170.3 \end{pmatrix} nA$$

$$Y \approx \frac{\Delta I}{\beta} = \begin{pmatrix} 0 \\ 1.31 \\ 2.49 \\ 3.8 \\ 4.7 \\ 6.01 \\ 7.2 \\ 8.5 \end{pmatrix}$$

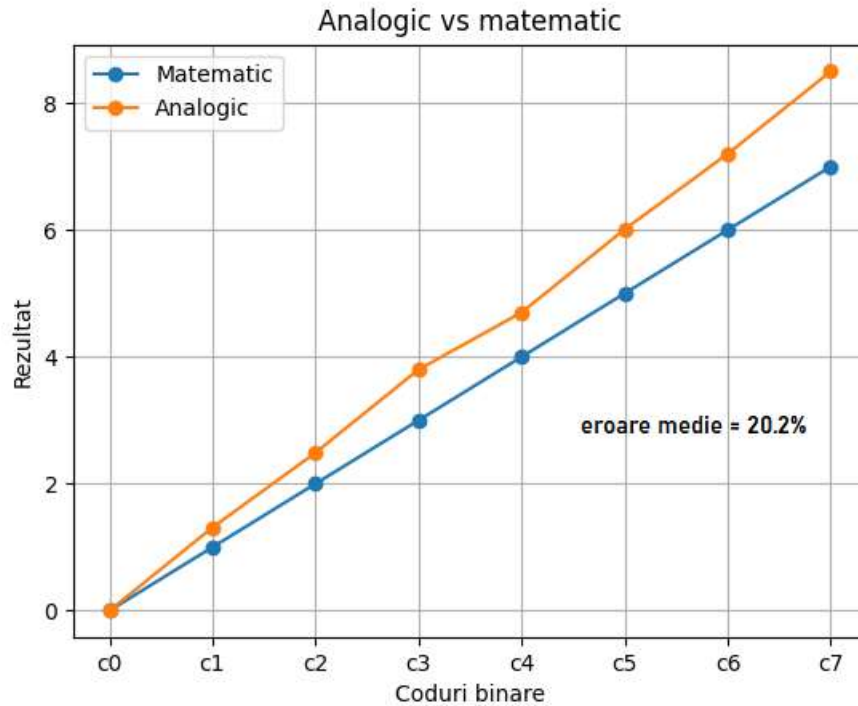


Fig.15

Aceasta eroare este de fapt o eroare de gain. Cu toate acestea, arhitectura isi pastreaza caracteristica liniara efectuand operatia de sume de produse corect.

Răspunsul în timp al unei operații de convoluție

Convoluția este definită așa cum am menționat anterior ca fiind operația efectuată între blocul kernel și un segment de dimensiune K din imaginea de intrare (a se vedea fig. 10).

Răspunsul arhitecturii la schimbarea valorilor de intrare și timpul de răspuns sunt esențiale în prelucrarea real-time a imaginilor. Am folosit pentru test un semnal de intrare corespunzător unui pixel de tip puls cu frecvența $f = 0.1\text{MHz}$, $t_{c,s} = 10\text{ns}$ și de amplitudine 50mV , valoare ce corespunde pixelului "1".



Fig. 16

Operatia se efectueaza intr-un timp de settle $t_s \approx 1 \mu s$. Valoarea de iesire in curent este cea asteptata $\Delta I \approx 26.22 nA$.

Pentru implementarea pur paralela, cel putin la nivel aproximativ, timpul de procesare este constant (procesare interna adica exclusiv operatia de convolutie). Daca in schimb, implementarea este cea hibrida, atunci timpul de procesare va fi direct proportional cu t_s , in functie de numarul de linii sau coloane din matricea de input. Cu toate acestea, este important de mentionat ca este posibil ca acest circuit sa fie prea lent, avand in vedere ca cel putin la iesirea sistemului trebuie facuta conversie analog-digital pentru a putea fi prelucrat semnalul.

Consumul static vs consumul operației de convoluție

O alta masura de performanta a circuitului este consumul de putere. Consumul unui MAC obisnuit este de ordinul pJ/MAC [5*]. Pentru a compara rezultate similare se face presupunerea ca esential este consumul dinamic, cel al operației in sine. Consumul static este și el important doar ca în posibile alternative de implementare, consumul static poate fi redus foarte mult prin activarea si dezactivarea celulelor in functie de operația făcută.

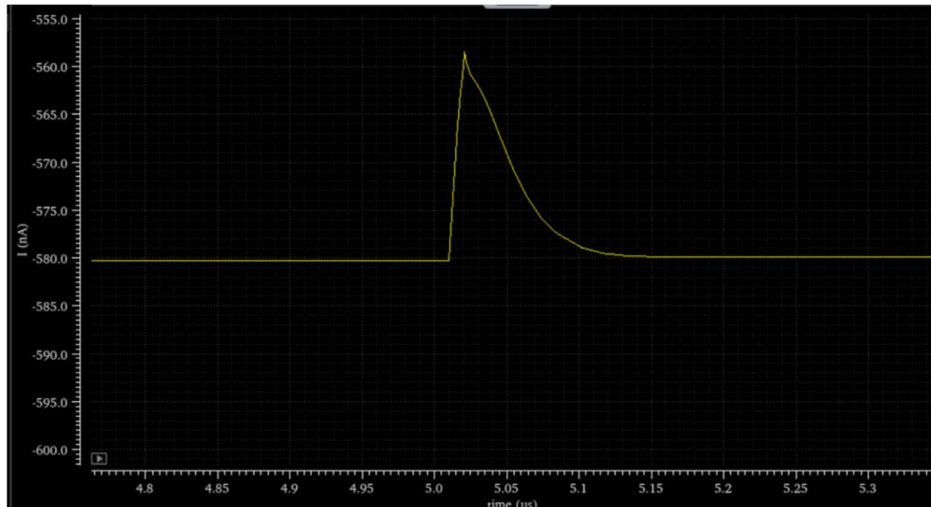


Fig. 17

curent consumat pentru 1 convolutie (9 celule active)

Consumul de curent static este in jur de 580 nA. Prin urmare, consumul de putere este $P_{\text{static}} \approx 1.044 \mu\text{W}$.

In ceea ce priveste consumul pentru fiecare celulă, acesta este semnificativ mai mic in regim dinamic. $P_{\text{celula}} = 4.39 \text{ nW}$, iar energia consumată in intervalul de calcul ($0.1 \mu\text{s}$) este: $E_{\text{celula/instantaneu}} = 0.439 \text{ fJ}$. Consumul total al unei celule in intervalul de calcul $t_s = 1 \mu\text{s}$ este: $E_t 116 \text{ fJ}$.

Pentru procesarea unei imagini in paralel (prima propunere de arhitectură) este nevoie de $(N - 2)^2 K^2$. Consumul estimat este:

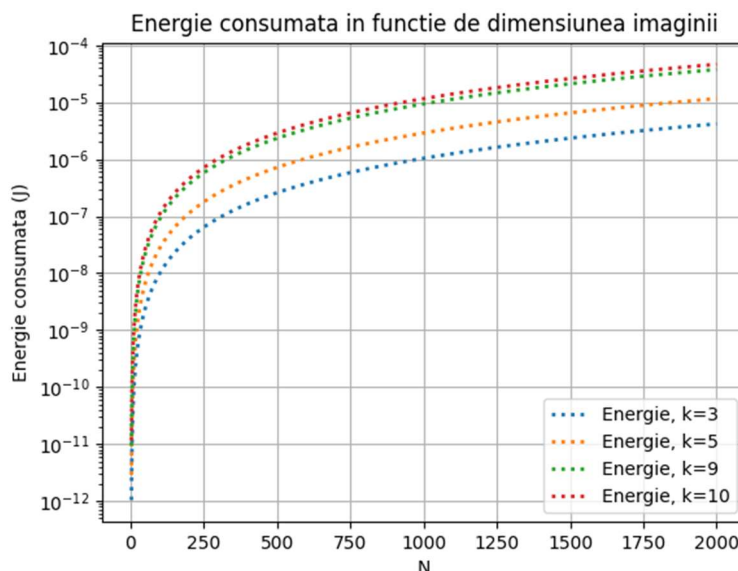


Fig. 18

Bibliografie

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning.*, Cambridge, MA, USA: MIT Press, 2016. P.II, Ch. 9

[2] B. Razavi, *Design of Analog CMOS Integrated Circuits*, McGraw-Hill, 2001, Chapter 2 (Weak Inversion)

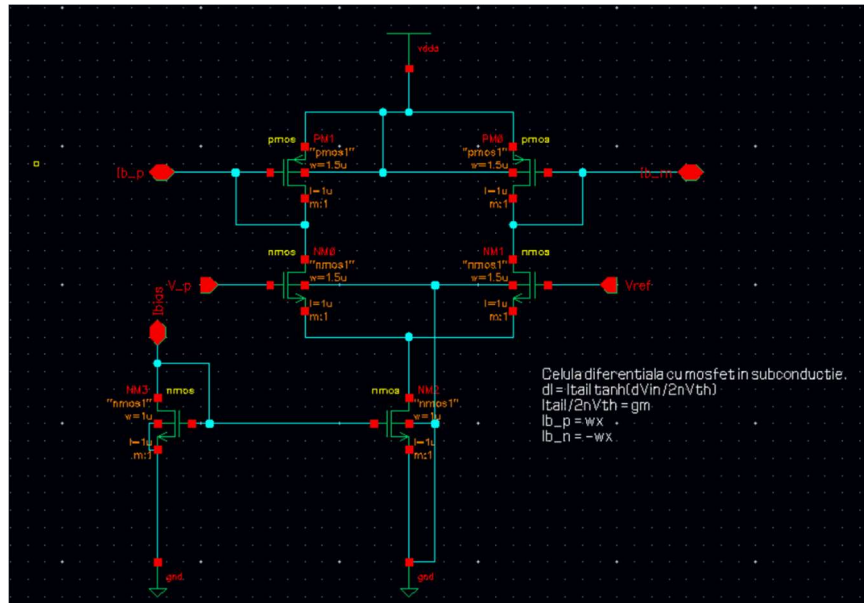
[3]<https://www.geeksforgeeks.org/deep-learning/kernels-filters-in-convolutional-neural-network/>

[4][Introduction to Convolution Neural Network - GeeksforGeeks](#)

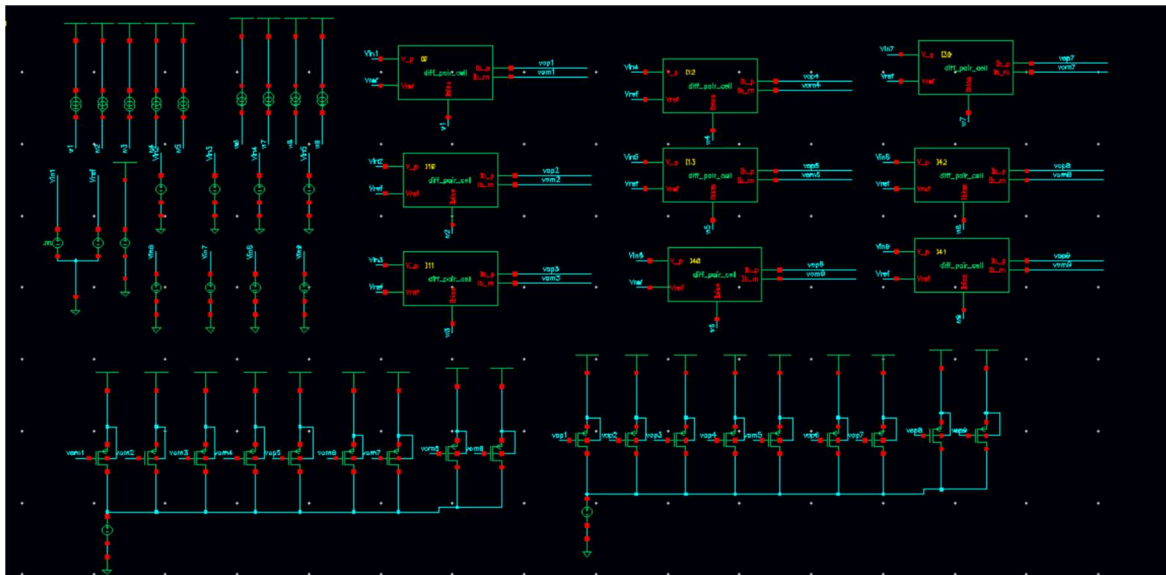
[5] **estimare aproximativa, simplificata pentru consumul unui MAC care are la baza articolul:** Mark Horowitz (Stanford) - “1.1 Computing’s Energy Problem”, 2014 IEEE International Solid-State Circuits Conference

Gemini. Am utilizat Google Gemini pentru clarificari matematice si exemple de cod Pyhon (realizarea graficelor in matplotlib), 2026.

Anexa



celula de calcul



arhitectura folosita in simulare